
User:

AWSに関する質問です。以下の3つのことを実現したいです。最新かつベストプラクティスで実現したいです。必ずAWSの公式ドキュメントから情報を探してかつ、ハルシネーションに十分に注意して実現方法を教えてください。よろしくお願いいたします。 - EventBridge を利用して平日の業務時間中（9:00～18:00）の間はEC2を起動し、それ以外の時間は停止するようにしたい - EventBridge を利用して平日の業務時間中（9:00～18:00）の間はECS Fargateを起動し、それ以外の時間は停止するようにしたい - ECS Fargate（ECS のタスク）は今後増える可能性があるので AWS CLI もしくは Lambda を利用する場合は PythonのAWS SDKを利用して起動しているタスク一覧を取得して、全タスクを停止させるような設定をしたい - EventBridge を利用して平日の業務時間中（9:00～18:00）の間はAWS RDSを起動し、それ以外の時間は停止するようにしたい

User:

それぞれ、CloudFormationテンプレートの作成例と必要であればPythonのサンプルコードを出力してほしいです。今回も必ずAWSの公式ドキュメントから情報を探してかつ、ハルシネーションに十分に注意して実現方法を教えてください。よろしくお願いいたします。できればAWS SDK の Reference ページや AWS CLIのReference ページを参考にしてほしいです。ECS Fargateの起動と停止は直接の回答の元になるドキュメントのページは存在しないかもしれませんが、色んな公式ドキュメントのページを参考にしながら実現方法を考えて出力してほしいです。

User:

実現したい「EventBridge を利用して平日の業務時間中（9:00～18:00）の間はECS Fargateを起動し、それ以外の時間は停止するようにしたい」で追加で調査してほしいです。調査してほしい内容としては、「EventBridge が指定の時間になったら、ECS Fargateを起動・停止をするLambdaを呼び出し、LambdaはPython, AWS SDK を利用して起動しているタスク一覧を取得して、全タスクを停止する処理を実施する」これを実現するための CloudFormation テンプレートとPythonのソースコードを出力してほしいです。今回も必ずAWSの公式ドキュメントから情報を探してかつ、ハルシネーションに十分に注意して実現方法を教えてください。よろしくお願いいたします。直接の回答の元になるドキュメントのページは存在しないかもしれませんが、色んな公式ドキュメントのページを参考にしながら実現方法を考えて出力してほしいです。

ChatGPT:

以下は **AWS公式ドキュメントをベースにした、EventBridge → Lambda → ECS Fargate タスク停止/起動を実現する CloudFormation テンプレート + Python コード** です。
要件は、「平日 9:00 に Fargate タスク起動」「平日 18:00 に起動中の全タスク停止」で、

EventBridge Scheduler から Lambda を呼び、Python (boto3) で ECS タスクをリストして停止します。

✦ 全体アーキテクチャ

1. EventBridge Scheduler

- 9:00（平日） → Fargate 起動用 Lambda を呼び
 - 18:00（平日） → Fargate 停止用 Lambda を呼び
- Scheduler は Lambda をトリガーする公式サポートがあります。* [AWS ドキュメント](#)

2. Lambda (Python)

- 起動 Lambda: run_task を boto3 で呼び
- 停止 Lambda: list_tasks → stop_task で全停止

3. IAM Role

- Scheduler が Lambda を呼びするための実行ロール
- Lambda が ECS へ boto3 でアクセスするための権限

✓ 1) CloudFormation テンプレート

※コメント付きで必要リソースを分かりやすく記載します。

```
yaml
AWSTemplateFormatVersion: "2010-09-09"
Description: >-
  Scheduled start/stop for ECS Fargate tasks on weekdays (9:00 start, 18:00 stop).

Parameters:
  EcsClusterName:
    Type: String
    Description: Target ECS cluster name
  EcsTaskDefinition:
    Type: String
    Description: Fargate task definition ARN or family:revision
  EcsSubnets:
    Type: List<AWS::EC2::Subnet::Id>
    Description: Subnets for Fargate awsipcConfiguration
  EcsSecurityGroups:
    Type: List<AWS::EC2::SecurityGroup::Id>
    Description: Security Groups for Fargate networking

Resources:

#####
# IAM Roles
#####
```

```

SchedulerInvokeRole:
  Type: AWS::IAM::Role
  Properties:
    AssumeRolePolicyDocument:
      Statement:
        - Effect: Allow
          Principal:
            Service: scheduler.amazonaws.com
          Action: sts:AssumeRole
    Path: "/"
    Policies:
      - PolicyName: "InvokeLambdaPolicy"
        PolicyDocument:
          Statement:
            - Effect: Allow
              Action: lambda:InvokeFunction
              Resource: "*"

# Role for Lambda to manage ECS
LambdaEcsRole:
  Type: AWS::IAM::Role
  Properties:
    AssumeRolePolicyDocument:
      Statement:
        - Effect: Allow
          Principal:
            Service: lambda.amazonaws.com
          Action: sts:AssumeRole
    Path: "/"
    ManagedPolicyArns:
      - arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole
    Policies:
      - PolicyName: "ECSControlPolicy"
        PolicyDocument:
          Statement:
            - Effect: Allow
              Action:
                - ecs:ListTasks
                - ecs:RunTask
                - ecs:StopTask
                - iam:PassRole
              Resource: "*"

#####
# ECS Start Lambda
#####

StartEcsLambda:
  Type: AWS::Lambda::Function
  Properties:
    Runtime: python3.9
    Role: !GetAtt LambdaEcsRole.Arn
    Handler: index.start_handler
    Timeout: 120
    Code:
      ZipFile: |
        import boto3
        import os
        ecs = boto3.client("ecs")
        def start_handler(event, context):
            cluster = os.environ["CLUSTER"]
            taskdef = os.environ["TASKDEF"]
            subnets = os.environ["SUBNETS"].split(",")
            sgs = os.environ["SECURITY_GROUPS"].split(",")

```

```

    ecs.run_task(
        cluster=cluster,
        taskDefinition=taskdef,
        launchType="FARGATE",
        networkConfiguration={
            "awsypcConfiguration": {
                "subnets": subnets,
                "securityGroups": sgs,
                "assignPublicIp": "ENABLED"
            }
        },
        count=1
    )
    return {"status": "started"}

```

```

Environment:
  Variables:
    CLUSTER: !Ref EcsClusterName
    TASKDEF: !Ref EcsTaskDefinition
    SUBNETS: !Join [",", !Ref EcsSubnets]
    SECURITY_GROUPS: !Join [",", !Ref EcsSecurityGroups]

```

```

#####
# ECS Stop Lambda
#####

```

```

StopEcsLambda:
  Type: AWS::Lambda::Function
  Properties:
    Runtime: python3.9
    Role: !GetAtt LambdaEcsRole.Arn
    Handler: index.stop_handler
    Timeout: 120
    Code:
      ZipFile: |
        import boto3
        import os
        ecs = boto3.client("ecs")
        def stop_handler(event, context):
            cluster = os.environ["CLUSTER"]
            # 全 RUNNING タスクを取得
            paginator = ecs.get_paginator("list_tasks")
            task_arns = []
            for page in paginator.paginate(cluster=cluster, desiredStatus="RUNNING"):
                task_arns.extend(page["taskArns"])
            # 全停止
            for task in task_arns:
                ecs.stop_task(cluster=cluster, task=task)
            return {"stopped_count": len(task_arns)}

```

```

Environment:
  Variables:
    CLUSTER: !Ref EcsClusterName

```

```

#####
# EventBridge Scheduler Rules
#####

```

```

StartSchedule:
  Type: AWS::Scheduler::Schedule
  Properties:
    Name: "Ecs-Fargate-Start-Weekdays"
    ScheduleExpression: "cron(0 9 ? * MON-FRI *)"
    ScheduleExpressionTimezone: "Asia/Tokyo"
    Target:

```

```

Arn: !GetAtt StartEcsLambda.Arn
RoleArn: !GetAtt SchedulerInvokeRole.Arn
FlexibleTimeWindow:
Mode: OFF

```

```

StopSchedule:
Type: AWS::Scheduler::Schedule
Properties:
Name: "Ecs-Fargate-Stop-Weekdays"
ScheduleExpression: "cron(0 18 ? * MON-FRI *)"
ScheduleExpressionTimezone: "Asia/Tokyo"
Target:
Arn: !GetAtt StopEcsLambda.Arn
RoleArn: !GetAtt SchedulerInvokeRole.Arn
FlexibleTimeWindow:
Mode: OFF

```

2) Python Lambda コード（引用元公式仕様）

Lambda の中身は 公式 **boto3** リファレンスに基づく呼び出し です。

■ 起動用ハンドラー

```

python
import boto3
import os

ecs = boto3.client("ecs")

def start_handler(event, context):
    cluster = os.environ["CLUSTER"]
    taskdef = os.environ["TASKDEF"]
    subnets = os.environ["SUBNETS"].split(",")
    sgs = os.environ["SECURITY_GROUPS"].split(",")

    ecs.run_task(
        cluster=cluster,
        taskDefinition=taskdef,
        launchType="FARGATE",
        networkConfiguration={
            "awsypcConfiguration": {
                "subnets": subnets,
                "securityGroups": sgs,
                "assignPublicIp": "ENABLED"
            }
        },
        count=1
    )
    return {"status": "started"}

```

- here uses `ecs.run_task` 公式 `boto3` API をそのまま呼んでいます。

■ 停止用ハンドラー

```
python
```

```
import boto3
import os

ecs = boto3.client("ecs")

def stop_handler(event, context):
    cluster = os.environ["CLUSTER"]
    paginator = ecs.get_paginator("list_tasks")
    task_arns = []
    for page in paginator.paginate(cluster=cluster, desiredStatus="RUNNING"):
        task_arns.extend(page["taskArns"])
    for task in task_arns:
        ecs.stop_task(cluster=cluster, task=task)
    return {"stopped_count": len(task_arns)}
```

stop_task は **boto3** リファレンスで正式サポートされており、Fargate タスク停止時の標準的 API です。 [Boto3](#)

✖ 補足（公式ドキュメントへのリンク）

操作	公式参照
EventBridge Scheduler で Lambda 呼び出し	Lambda + Scheduler 公式ドキュメント AWS ドキュメント
ECS タスク起動	ECS の run_task API （boto3）
ECS タスク停止	stop_task boto3 API リファレンス Boto3

📌 注意ポイント

- **IAM 権限**：Scheduler → Lambda 呼び出しは IAM 実行ロールが必要です（テンプレ内 IAM Role）。 [AWS ドキュメント](#)
- **ECS networking**：Fargate は awsvpcConfiguration でサブネットと SG を設定する必要があります。
- **Scheduler の Cron**：タイムゾーン指定を Asia/Tokyo にして日本時間で稼働。

必要であれば、**サービス単位のスケーリング停止/起動パターン（Service DesiredCount を変更）** 版も作成できます。希望する場合はお知らせください。